Universitatea POLITEHNICA din București
Facultatea de Automatică și Calculatoare

Electronică digitală

Curs – Modulul 9

Multiplexare și acces concurent

dr. ing. Răzvan Tătăroiu
razvan.tataroiu@upb.ro

2022-2023

Sumar

- Definiții
- Porți și structuri logice specifice
  - Multiplexoare digitale
  - Comutatoare și multiplexoare analogice
  - Porți cu 3 stări
  - Porți cu colector / drenă în gol
- Aplicații

Multiplexare – definiție generală

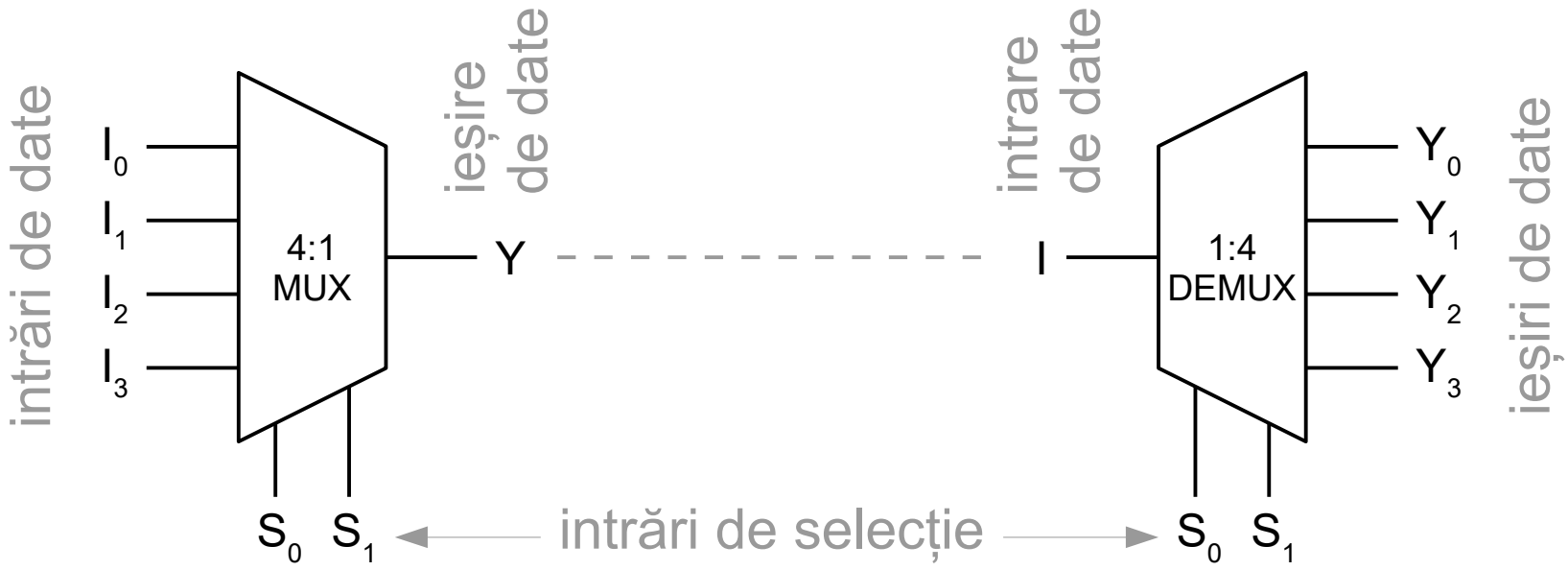
- Multiplexare
  - în sens larg: tehnică prin care o singură resursă deservește mai multe canale (este partajată)
- Exemple:
  - în telecomunicații: pe un singur cablu (electric sau optic) se transmit simultan mai multe convorbiri telefonice sau în general fluxuri de date (*streams*)
  - în achiziția datelor: cu un singur convertor analog-digital (ADC) se citesc secvențial mai multe canale de măsură

Multiplexare
(simplă)

Multiplexare – definiție restrânsă

- La nivel de circuit, de obicei:
  - un **multiplexor** selectează unul din multiplele semnale de intrare și îl conectează sau reproduce pe unica ieșire
  - un **demultiplexor** selectează una din multiplele ieșiri, unde conectează sau reproduce unicul semnal de intrare
  - strâns legat de acestea este **decodorul**: din cele 2^n ieșiri, o activează doar pe cea cu numărul reprezentat în baza 2 pe cele *n* intrări de selecție

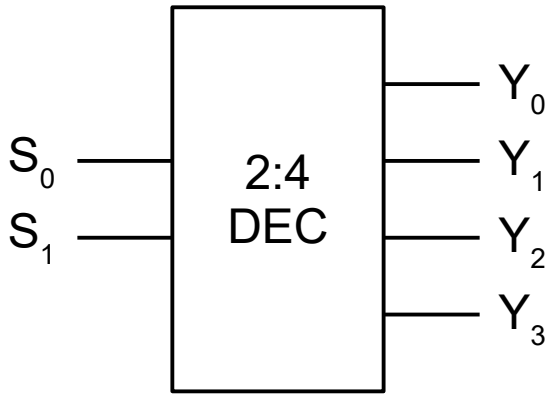
Multiplexor, demultiplexor



Multiplexor
(abreviat MUX)
ex.: 74HC153

Demultiplexor
(abreviat DEMUX)
ex.: 74HC139

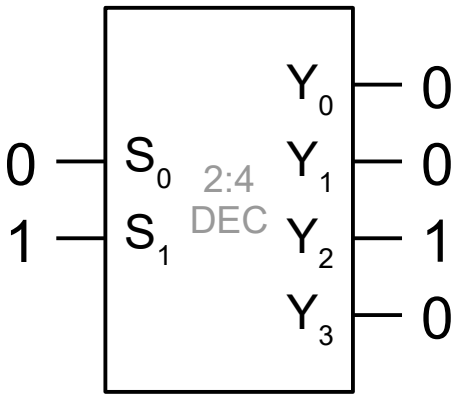
Decodor



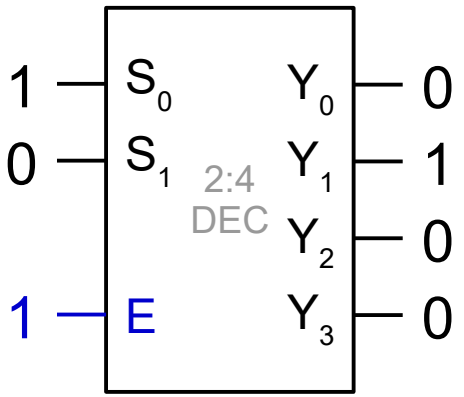
Intrările de selecție reprezintă în baza 2 un număr $s = 2S_1+S_0$

Ieșirea Y_S va fi activă (“logic 1”), iar celelalte ieșiri vor fi inactive (“logic 0”).

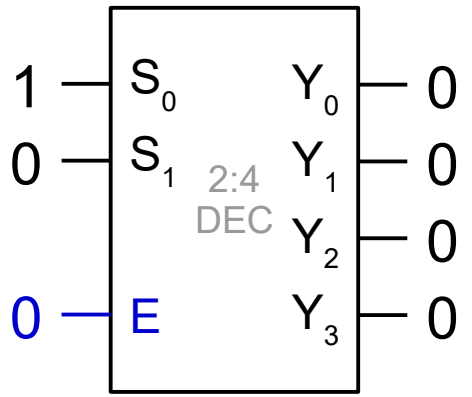
Exemplu:



OBS.: Există decodoare cu una sau mai multe intrări de activare (***E**nable*):

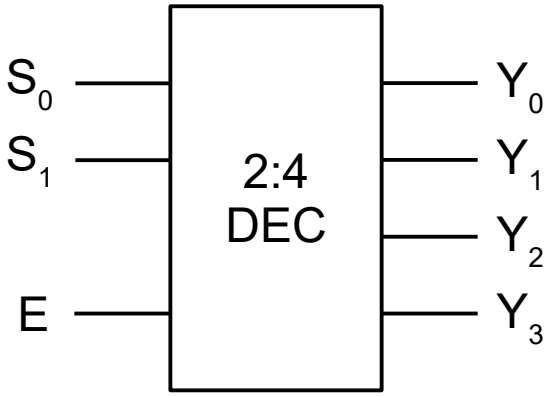


Enable activ:
funcționează ca DEC

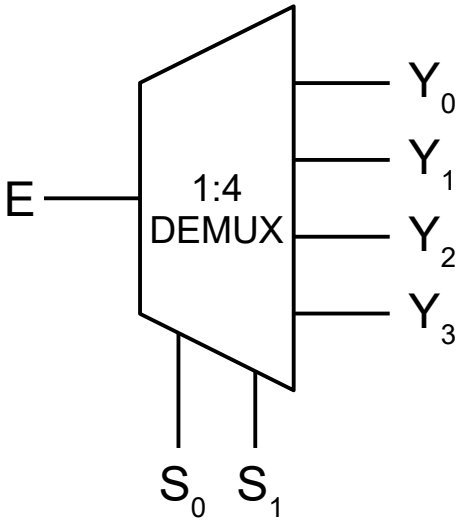


Enable inactiv:
toate ieșirile inactive

Decodor / demultiplexor



Decodor



Demultiplexor

Observație:
La nivel boolean, un **demultiplexor** $1:2^n$ este practic identic cu un **decodor** $n:2^n$ cu datele aplicate pe intrarea de **E**nable.
(comportament identic; vezi și ecuațiile de pe planșa următoare)

Exemplu: 74HC139 (DEC / DEMUX având E și ieșirile negate)

Implementare DEC/DEMUX cu porți

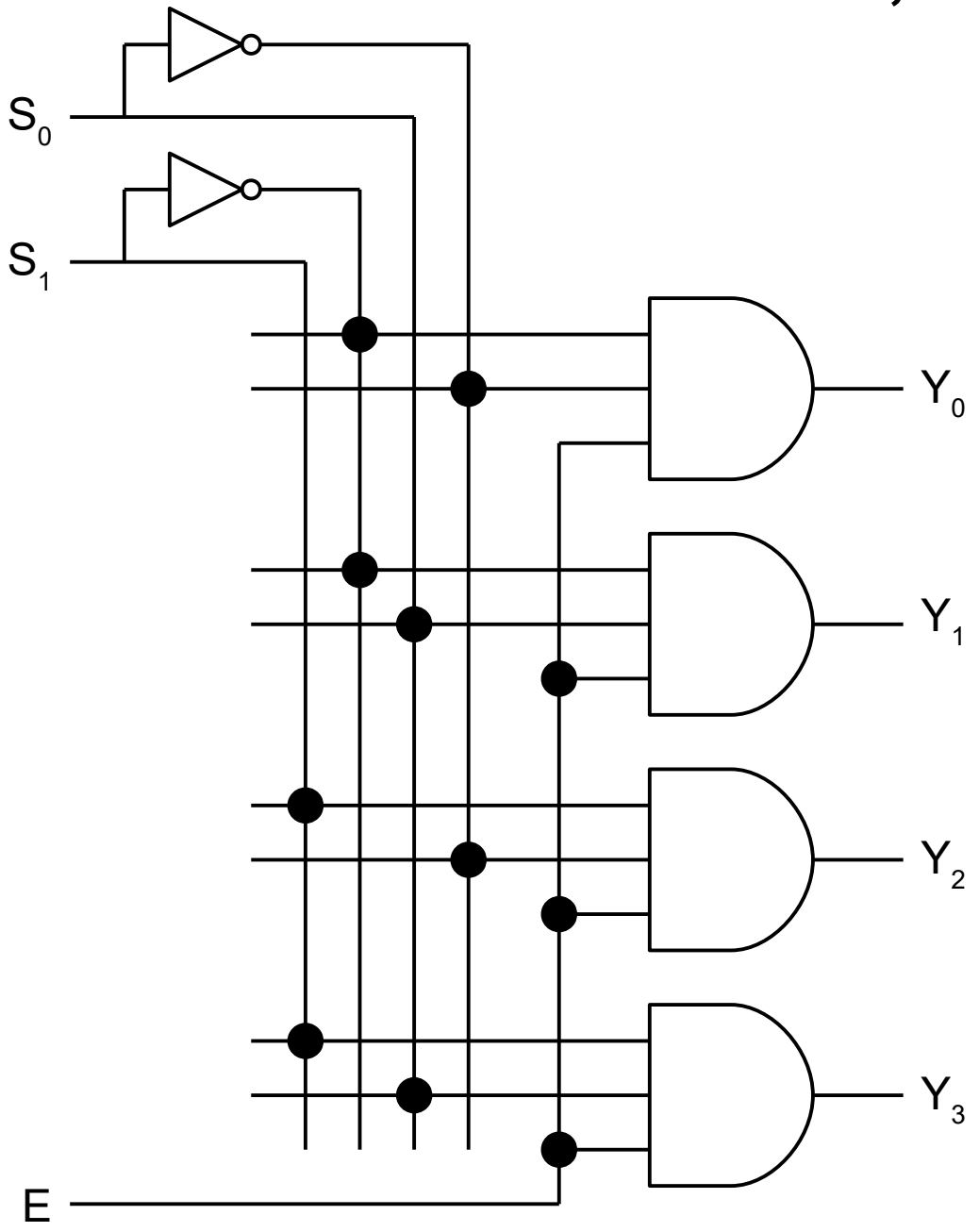
$$Y_0 = E\,\overline{S_1}\,\overline{S_0}$$

$$Y_1 = E\,\overline{S_1}\,S_0$$

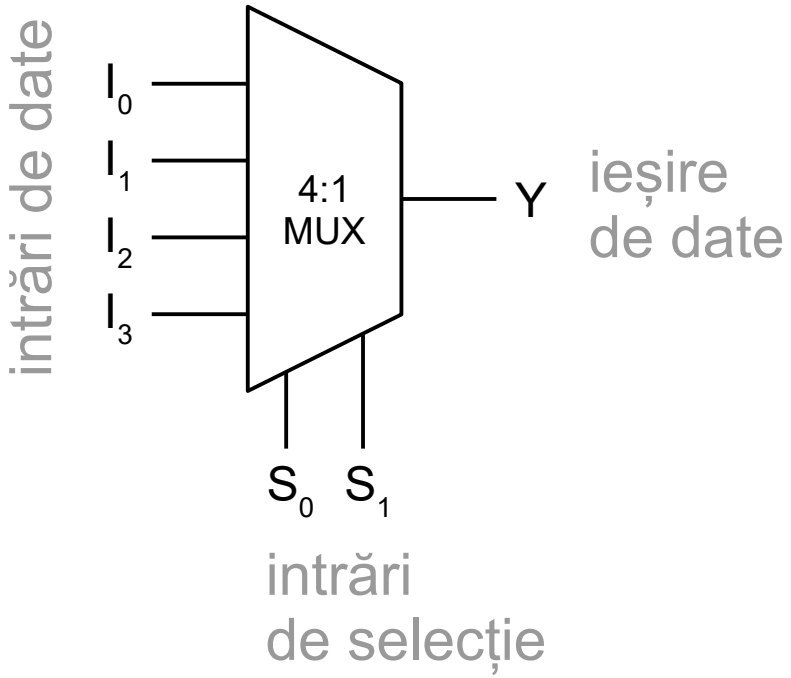
$$Y_2 = E\,S_1\,\overline{S_0}$$

$$Y_3 = E\,S_1\,S_0$$

Observație generală
privind implementarea funcțiilor boolene cu matrice de conectare: factorii care intră în ecuație corespund conexiunilor realizate între liniile verticale (factori posibili) și cele orizontale (intrări în porți ȘI)
Există circuite programabile în acest sens – *keywords: Programmable Logic Device*



MUX – funcție booleană



Descriere comportamentală:
Y preia valoarea intrării I_s, unde
s este numărul codificat de intrările de selecție S_1, S_0 ($s = 2S_1 + S_0$)

Descriere booleană:

$$Y = Y_0 \vee Y_1 \vee Y_2 \vee Y_3$$ SAU logic

unde:

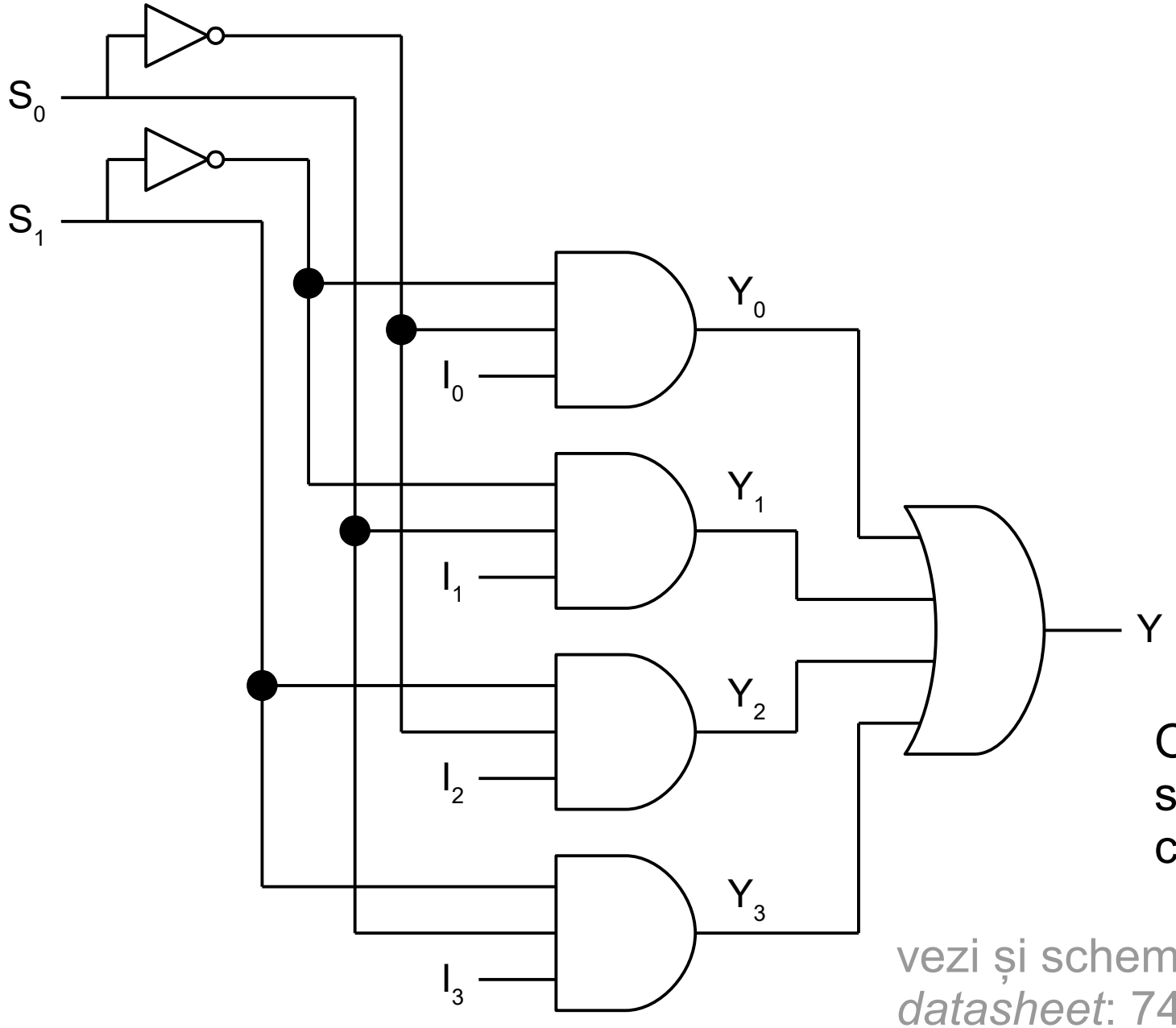
$$Y_0 = I_0 \overline{S_1}\,\overline{S_0}$$

$$Y_1 = I_1 \overline{S_1} S_0$$

$$Y_2 = I_2 S_1 \overline{S_0}$$

$$Y_3 = I_3 S_1 S_0$$

Implementare MUX cu porți



$$Y_0 = I_0 \overline{S_1}\,\overline{S_0}$$

$$Y_1 = I_1 \overline{S_1} S_0$$

$$Y_2 = I_2 S_1 \overline{S_0}$$

$$Y_3 = I_3 S_1 S_0$$

$$Y = Y_0 \vee Y_1 \vee Y_2 \vee Y_3$$

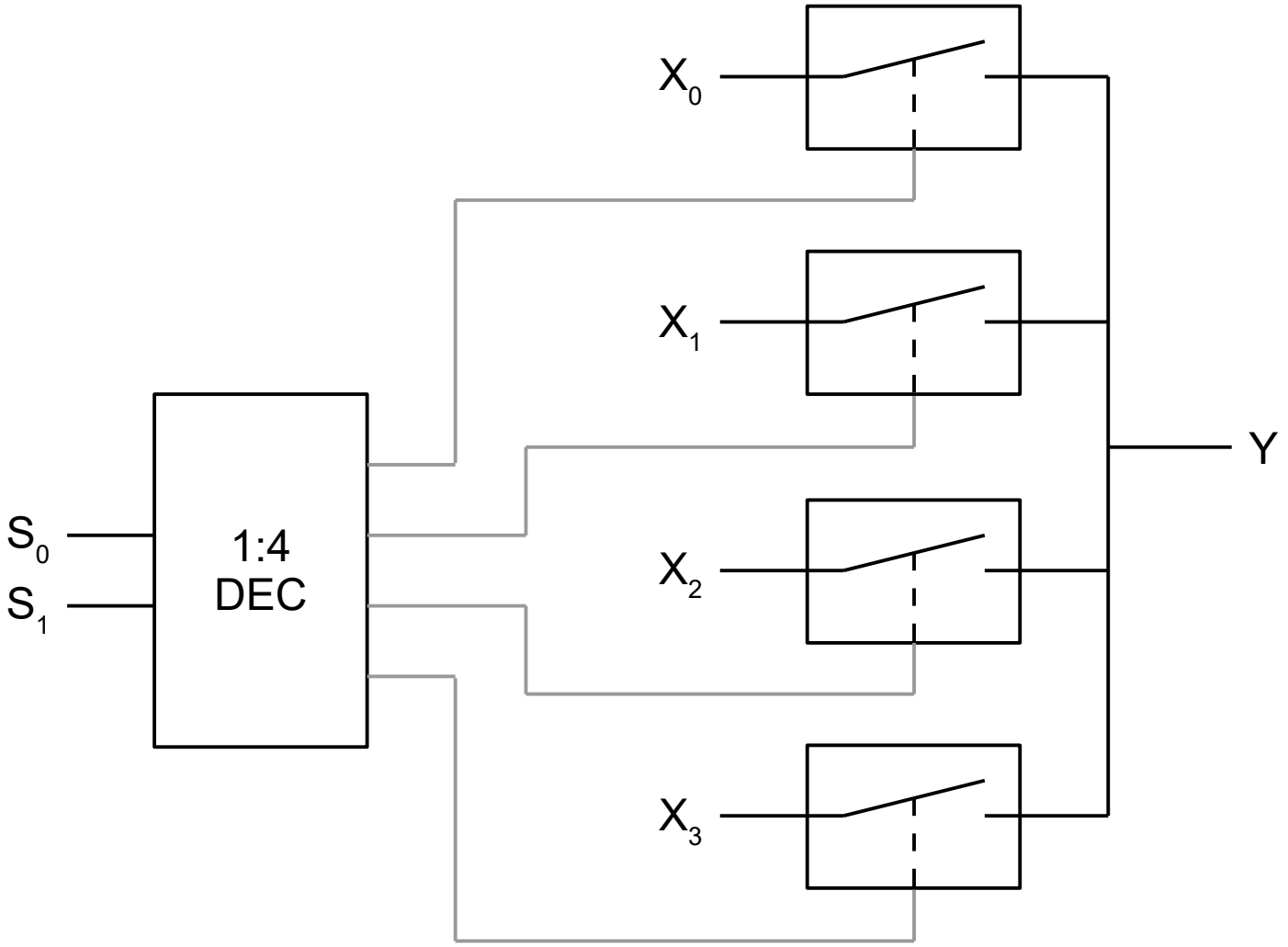
OBS. ecuațiile și schema se aseamănă cu DEC / DEMUX

vezi și schemele la nivel de porți din *datasheet*: 74HC153 sau 74HC151.

MUX / DEMUX cu comutatoare (1)

- Comentarii
  - implementarea **booleană** (cu porți) **reproduce** intrarea selectată pe ieșire
  - la fel în cazul DEMUX, unde intrarea este reprodusă pe ieșirea selectată
  - într-o implementare mai apropiată de descrierea **comportamentală**, un decodor comandă 2^n comutatoare, cel selectat **conectând direct** intrarea vizată la ieșire
  - comutatoarele fiind bidirecționale, funcționează la fel de bine și ca **de**multiplexor

MUX / DEMUX cu comutatoare (2)



OBS.: X_s și Y pot fi utilizate atât ca intrări cât și ca ieșiri.

exemple: 74HC4051 (8:1), 74HC4052 (4:1)

MUX / DEMUX cu comutatoare (3)

- Întrucât schema cuplează direct terminalul comun (Y) cu terminalul selectat (X_s), acestea nu prezintă în mod special caracter de intrare sau de ieșire (ca în situația implementării cu porți logice)

- Astfel dispozitivul poate fi folosit:
  - ca multiplexor sau demultiplexor
  - atât pentru semnale digitale cât și analogice

MUX / DEMUX cu comutatoare (4)

- Funcționează atât cu semnale digitale cât și **analogice**
  - motiv pentru care mai este numit **multiplexor analogic**
  - iar comutatoarele din componența sa – **comutatoare analogice**
    - există și ca piese de sine stătătoare (comutatoare controlabile individual – ex. 74HC4066)
    - denumite și **porți de transmisie** pentru că permit transmiterea în ambele sensuri a semnalelor

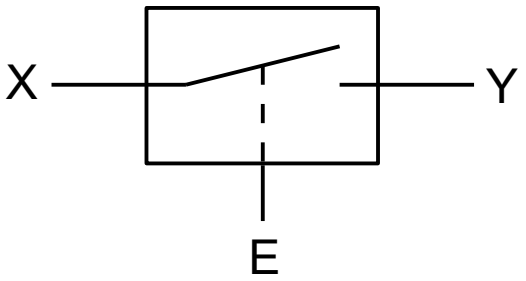
Comutatoare și MUX-uri analogice

- Aplicații
  - comutarea și multiplexarea semnalelor **digitale**
  - comutarea și multiplexarea semnalelor analogice
  - circuite analogice configurabile digital
    - amplificatoare cu câștig programabil (*Programmable Gain Amplifiers*)
    - filtre cu capacități comutate (*Switched Capacitor Filters*)
- Utilizarea în aplicații analogice (cu cerințe de precizie) impune analiza influenței parametrilor electrici ai acestor comutatoare (neidealitățile lor). Multe sunt clasificate de producători în mod implicit ca piese digitale (imprecise).

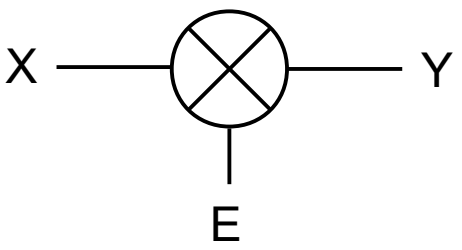
Comutator analogic – sumar

- denumit și **poartă de transmisie**
  - deși spre deosebire de porțile logice, care refac nivelele *low/high* ale semnalului, aceasta ideal nu îl influențează
- comutator electronic (nu mecanic)
  - comandat de un semnal digital (***E**nable*)
- simboluri (echivalente):

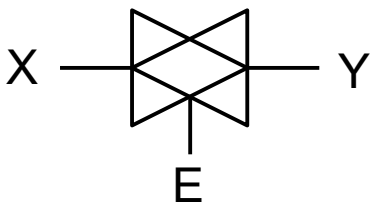
"comutator comandat"



"Y = X * E" ?



"poartă de transmisie bidirecțională"
- impropriu, întrucât nu amplifică semnalul

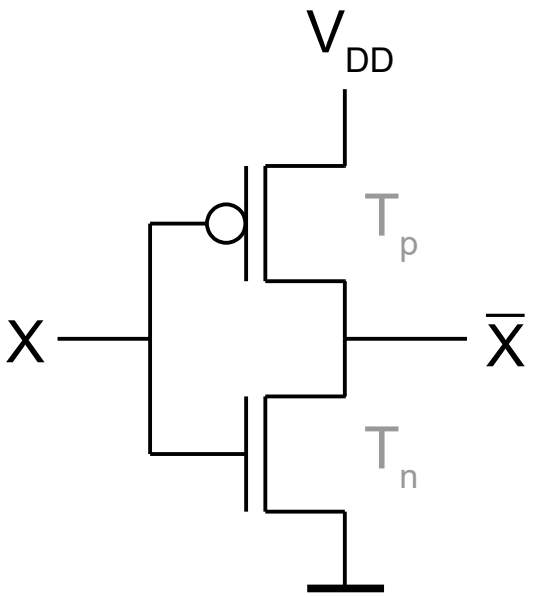


Comutator analogic – implementare

- cu tranzistoare bipolare
  - de obicei **nu**, deoarece prezintă neidealități: curent de bază, tensiune reziduală C-E la saturație
- cu tranzistoare MOS
  - comportament rezonabil – aproximat de un comutator ideal în serie cu o rezistență (neliniară, dar mică)
  - implementare CMOS – combinație paralel între:
    - un tranzistor NMOS care conduce bine semnalele apropiate de masă (în porțile logice “trage jos”)
    - un tranzistor PMOS care conduce bine semnalele apropiate de tensiunea de alimentare (“trage sus”)

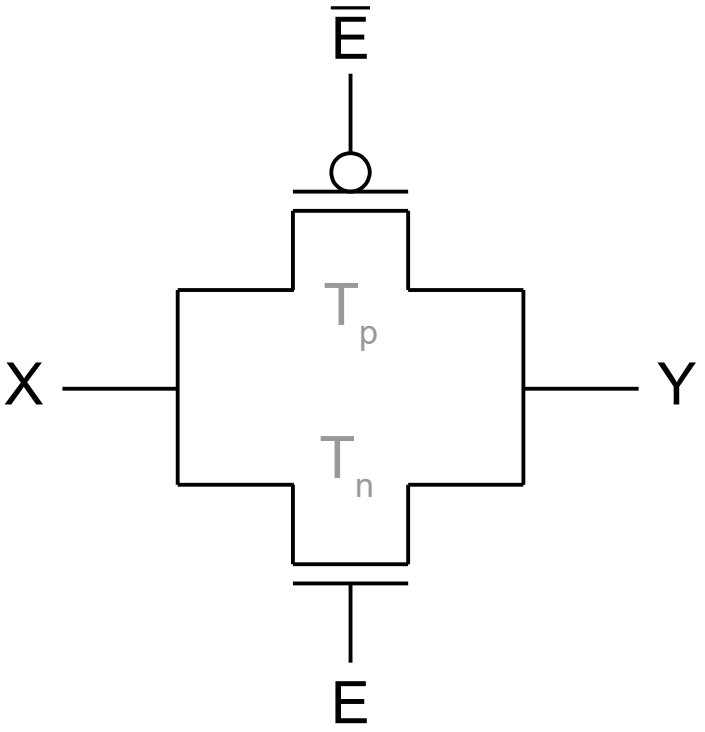
Comutator analogic CMOS

Poartă NOT,
pentru comparație



Tranzistoarele
conduc **alternativ**
în funcție de X

Comutator analogic



Tranzistoarele
conduc **simultan**
sau niciunul

Comutatoare analogice – precauții

- diferențe notabile față de comutatoarele mecanice:
  - curent maxim tolerat foarte mic (mA)
  - rezistență-serie de ordinul Ω ... kΩ, neliniară, dependentă de tensiunea semnalului comutat – nu contează la curent foarte mic; ideal curentul prin comutatoarele analogice trebuie să fie **zero** – să fie legate spre o intrare digitală CMOS / intrare de amplificator operațional
  - tensiunea semnalului limitată – între tensiunile de alimentare (ex.: **nu** putem comuta semnale în domeniul -10...+10 V cu un comutator alimentat între 0 și 5 V. Acesta ar trebui alimentat de ex. la -12 și +12 V dacă permite – trebuie ținut cont de limitele din *datasheet*)
- pentru tensiuni și curenți mari se folosesc **relee *solid-state*** (cu intrarea digitală izolată și ieșire specifică de c.c. sau c.a.)

Acces concurent

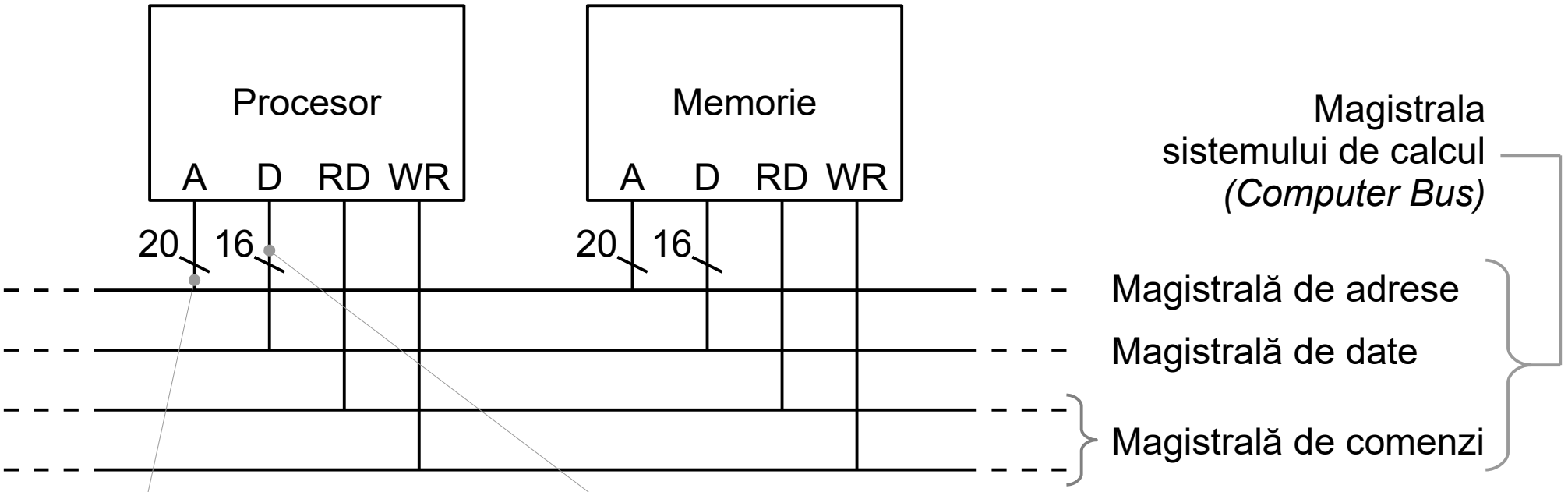
Acces concurent – definiție

- Acces concurent: formă de multiplexare cu caracter complex, ce implică un mecanism de control al accesului la o resursă partajată
- Exemple:
  - multiple dispozitive (“stații”) doresc să transmită simultan date într-o rețea *wireless*
  - un microprocesor trebuie să comunice cu mai multe periferice (senzori, actuatoare, afișaj etc.) conectate la aceeași **magistrală** (*computer bus*)
  - multiple dispozitive conectate la o magistrală doresc să semnaleze prezența unor noi date (pentru ca procesorul să solicite să le citească)

Arhitectura clasică de calculator

- Vom ilustra conceptul de acces concurent în cadrul arhitecturii clasice de calculator
- Componente principale:
  - un microprocesor (μP), denumit și *Central Processing Unit (CPU)*
  - o memorie cu acces aleator (*Random Access Memory – RAM*)
  - diverse dispozitive periferice
    - tastatură, *mouse*, *touchscreen*, afișaj, interfață rețea etc.
  - o magistrală ce conectează elementele de mai sus

CPU, RAM și magistrală



20 linii de **a**dresă reprezentate grupat
(în general pot fi oricâte)
=> pot reprezenta $2^{20} \approx 10^6$ (1M) adrese

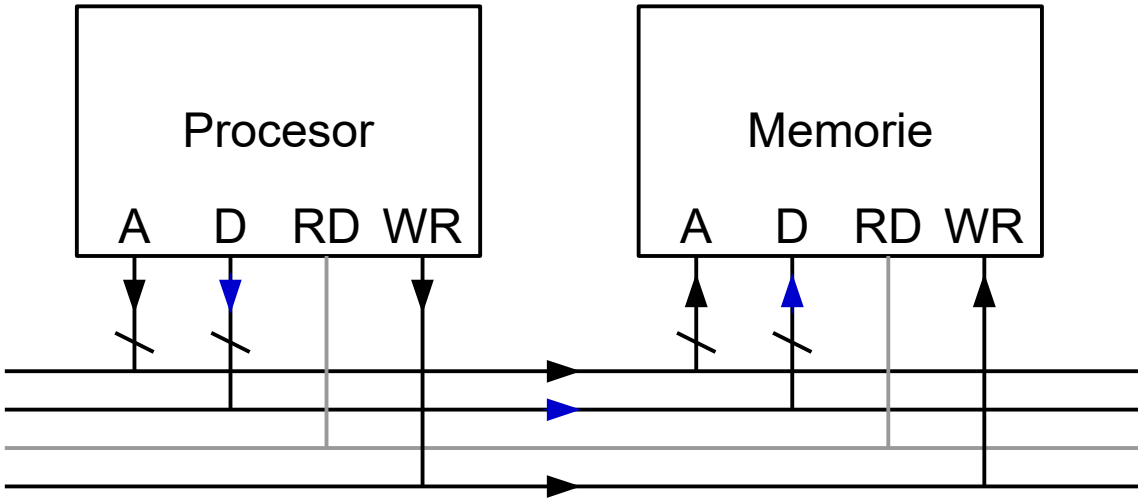
16 linii de **d**ate reprezentate grupat
(oricâte, de obicei putere a lui 2)
=> între procesor și locația de memorie adresată se pot transfera simultan 16 biți (un "cuvânt" - *word*)

RD: comandă de citire – determină un transfer de date dinspre memorie spre procesor
WR: comandă de scriere – determină un transfer de date dinspre procesor spre memorie

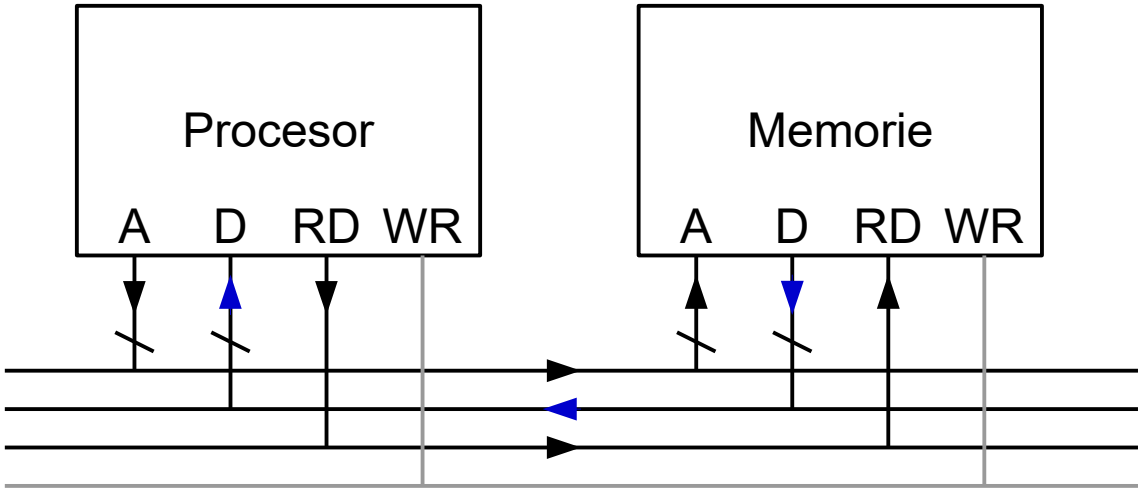
Transferul de date pe magistrală (1)

- Procesorul controlează toate operațiile de transfer de date (citiri și scrieri) – *bus master*
- Scriere
  - procesorul emite **a**dresa unde vrea să scrie pe liniile **A**
  - procesorul emite **d**atele pe liniile **D**
  - procesorul activează semnalul **WR**ite
  - memoria preia datele
- Citire
  - procesorul emite **a**dresa de unde vrea să citească pe liniile A
  - procesorul activează semnalul **R**ea**D**
  - memoria emite **d**atele aflate la adresa A pe liniile **D**, de unde procesorul le preia

Transferul de date pe magistrală (2)



Operație de scriere în memorie



Operație de citire din memorie

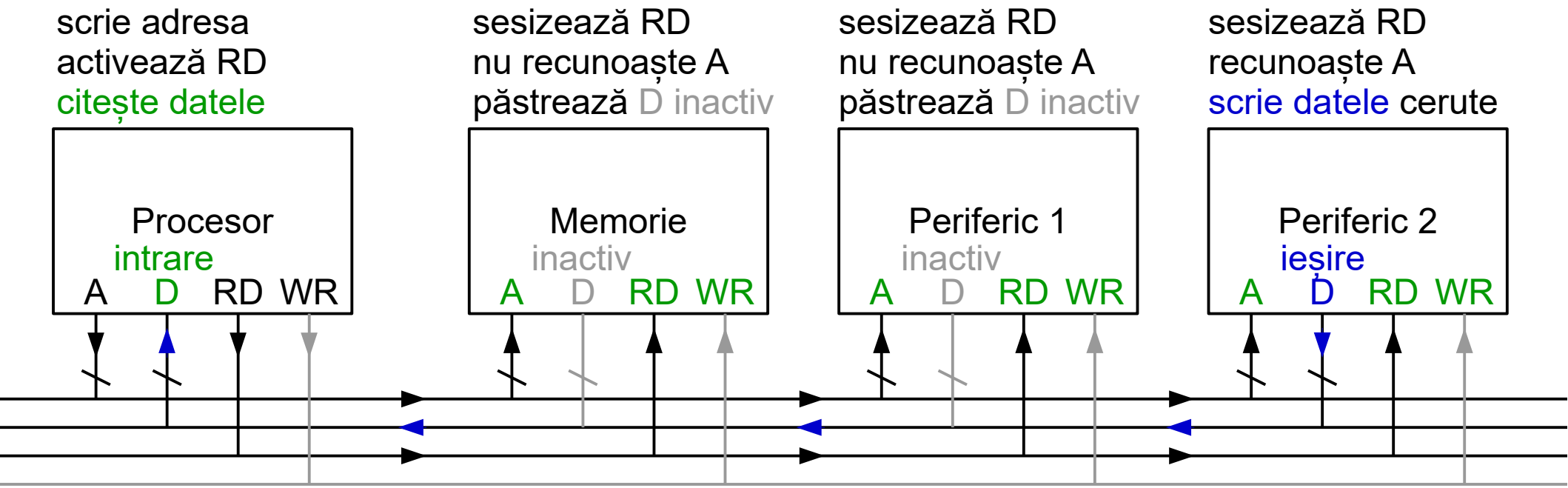
Observații:

- întotdeauna procesorul emite adrese și comenzi (liniile A, RD și WR sunt întotdeauna **ieșiri** din procesor și **intrări** în memorie)
- la scriere / citire liniile **D își schimbă sensul** (din ieșiri devin intrări și invers)

Detalii

- În realitate pe magistrală există, alături de memorie, multiple dispozitive periferice, cu care procesorul schimbă date în același mod
  - spațiul de memorie al procesorului este partiționat între memoria propriu-zisă și dispozitivele periferice
- Fiecare dispozitiv de pe magistrală (memorie sau periferic) “vede” toate operațiile solicitate de procesor, dar le efectuează doar pe cele la care își recunoaște adresa
  - de exemplu făcând AND între biții semnificativi ai adresei (direct sau negați corespunzător)

Transfer de date cu un periferic



Concluzie – din punct de vedere electric:
fiecare dispozitiv trebuie să poată, pe aceleași linii de date, după caz:
* să citească (caracter de intrare)
* să scrie (caracter de ieșire).
Caracterul de **ieșire** al terminalelor respective trebuie să poată fi **inactivat** pentru a permite altui dispozitiv să scrie date.

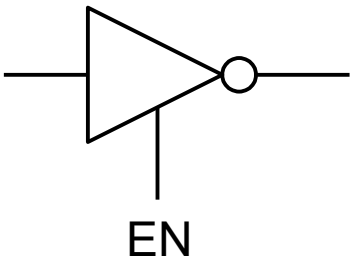
OBS.: Sistemul va fi proiectat a.î. niciodată două dispozitive să nu recunoască simultan o adresă (conflict la scrierea datelor pe magistrală)

Porți cu 3 stări – sumar

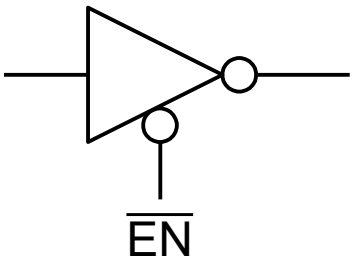
- Stări:
  - active: **L**ow, **H**igh (cele obișnuite)
  - inactivă: **Z** (de mare impedanță) – poarta nu “trage” ieșirea nici în jos nici în sus, presupunându-se că există legată în paralel altă ieșire de poartă care este activă. Ieșirea inactivă este similară unui comutator oprit – are impedanță foarte mare.
- Utilitate:
  - acces concurent la magistrale de date

Porți cu 3 stări – comportament

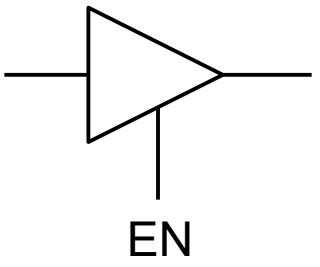
- Porțile cu 3 stări sunt dotate, pe lângă intrările normale, cu o intrare specială de activare (**EN**able). Când aceasta este activă, poarta funcționează normal. Când este inactivă, ieșirea porții este oprită (este în stare **Z**).



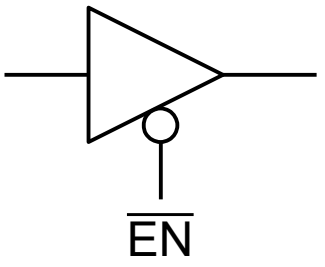
inversor cu 3 stări
cu EN activ pe 1



inversor cu 3 stări
cu EN activ pe 0



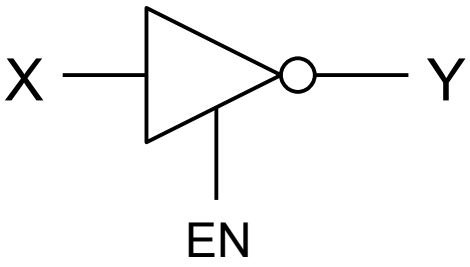
repetor cu 3 stări
cu EN activ pe 0

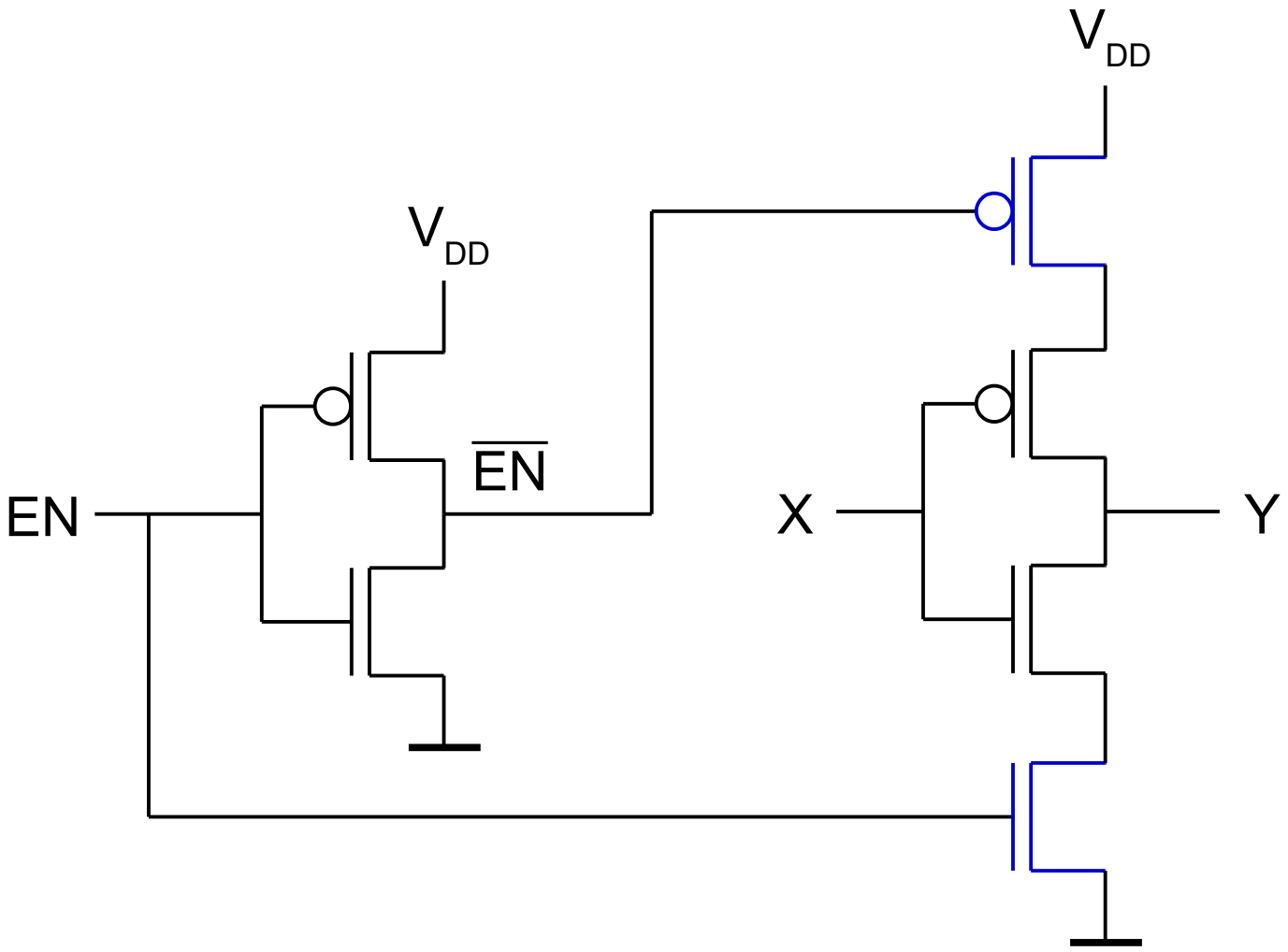


repetor cu 3 stări
cu EN activ pe 0

Porți cu 3 stări – schemă CMOS

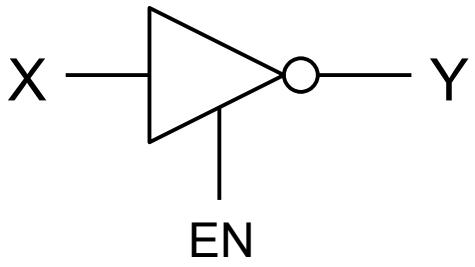
Varianta cu oprirea explicită a ieșirii
(tranzistoare în serie cu cele din poarta normală)





Porți cu 3 stări – schemă CMOS

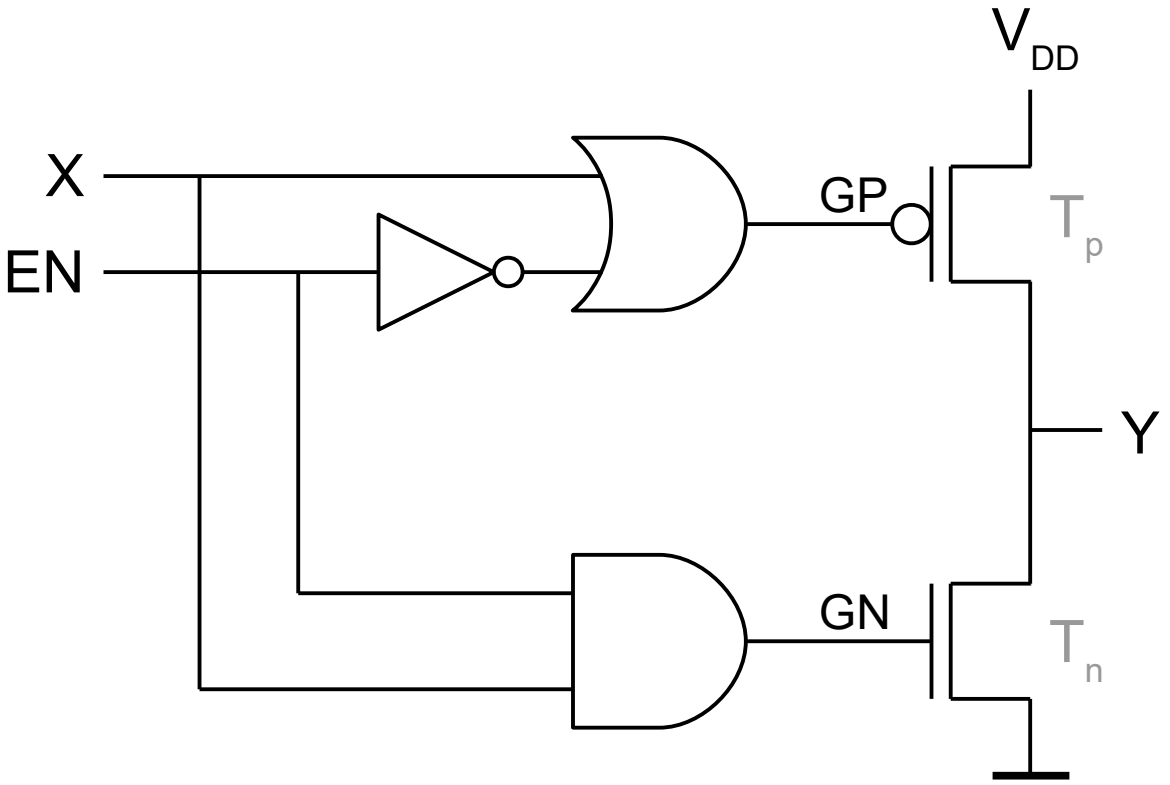
Varianta cu comanda individuală a tranzistoarelor de ieșire
avantaj: doar 2 tranzistoare mari (pe ieșire) în loc de 4



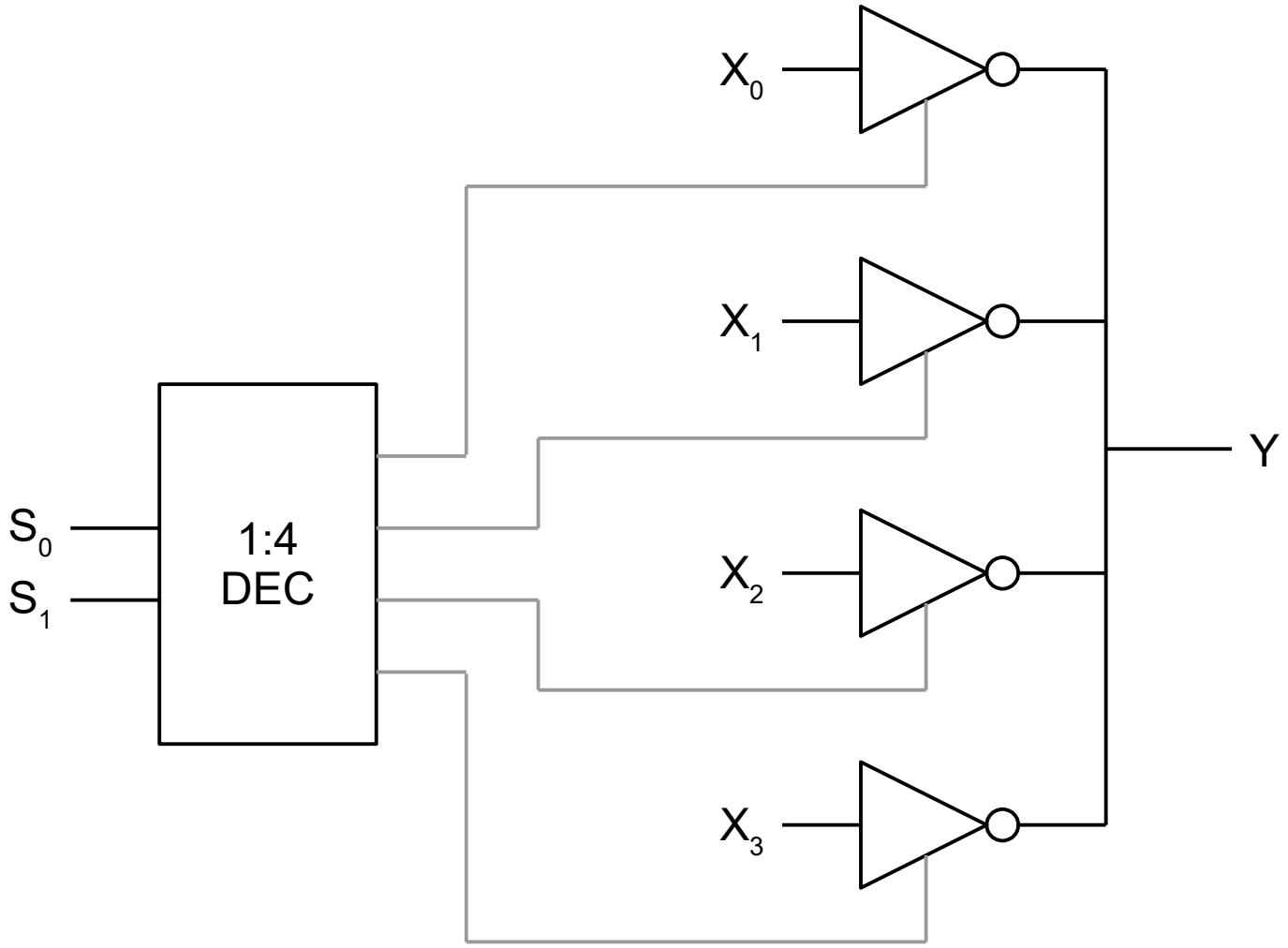
EN	X	Y	Tn	GN	Tp	GP
0	0	Z	OFF	0	OFF	1
0	1	Z	OFF	0	OFF	1
1	0	1	OFF	0	ON	0
1	1	0	ON	1	OFF	1

$$GN = EN \cdot X$$

$$GP = \overline{EN} \vee X$$



MUX cu porți cu 3 stări



Porți cu 3 stări – comentarii

- Sunt proiectate în vederea conectării mai multor ieșiri în paralel, cu condiția ca să nu fie active mai multe ieșiri simultan
  - fapt asigurat de decodorul din exemplul anterior sau de alocarea adreselor pe magistrale
- Sunt situații în care nu se poate asigura excluderea activării simultane
  - exemplu: **linie de întrerupere** partajată
  - atunci nu se pot utiliza porți cu 3 stări
  - se pot însă utiliza structuri tip ȘI-cablat

Ce este o întrerupere?

- Un semnal emis de un periferic către procesor
- Semnalează un eveniment ce necesită atenția programului ce rulează pe procesor
  - de obicei indică faptul că perifericul are disponibile date noi pe care procesorul trebuie să le citească, spre exemplu:
    - a fost recepționat un pachet de date pe rețea
    - utilizatorul a apăsat o tastă / a făcut un gest
  - alt exemplu:
    - întrerupere de *timer* – periodică sau când se scurge un interval de timp programat

De ce se utilizează întreruperi?

- Pentru că altfel ar trebui ca procesorul să interogheze periodic toate perifericele
  - ineficient, consumă energie degeaba
  - procesorul trebuie totuși să execute și alte sarcini, ceea ce înseamnă că perifericele nu pot fi interogate la momente precise, ci doar când execuția programului ajunge în punctul respectiv

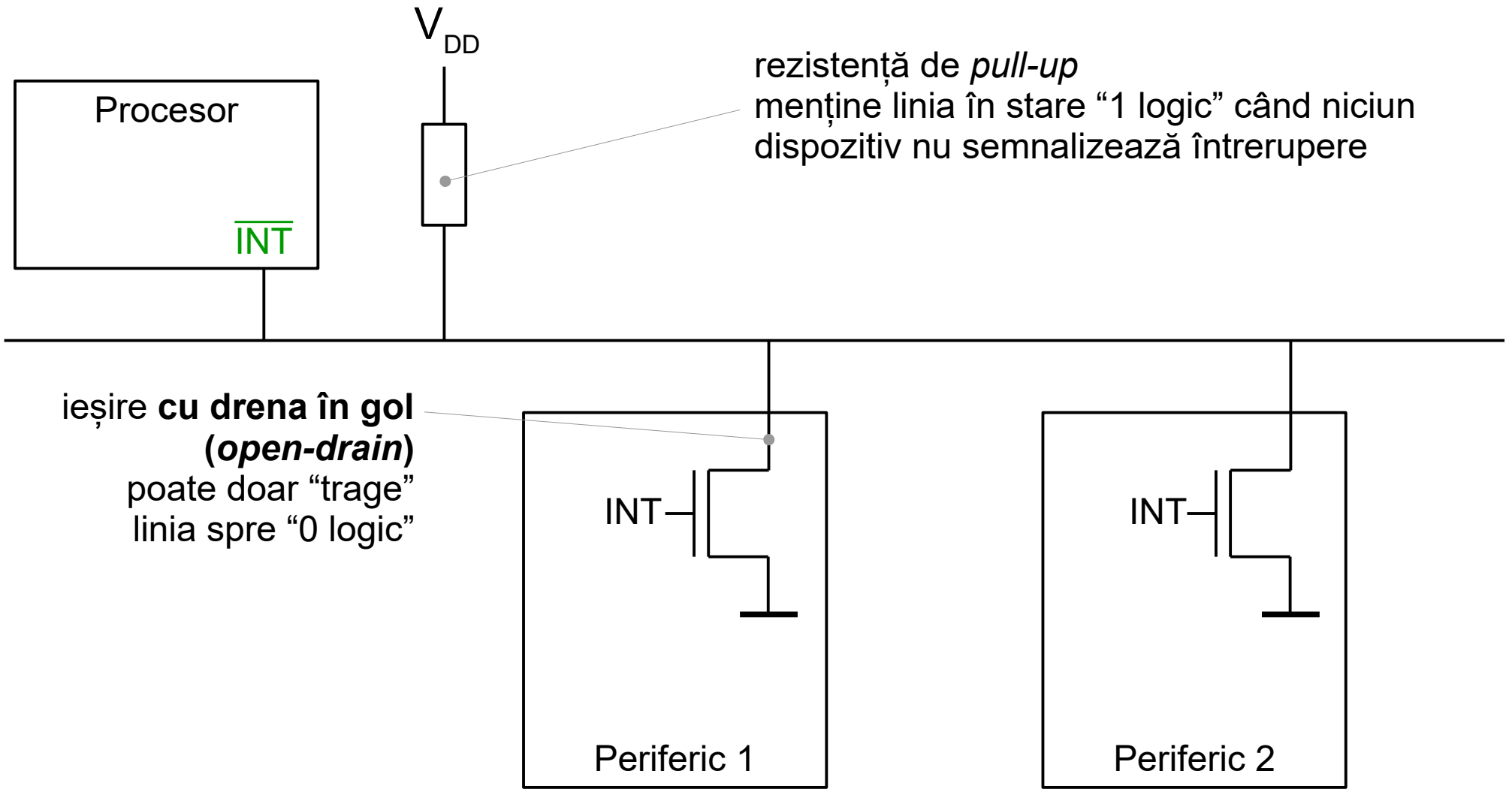
Cum funcționează întreruperile?

- Perifericul ce solicită atenția procesorului activează linia de întrerupere
  - spre deosebire de celelalte operații, care sunt întotdeauna inițiate de procesor
- Procesorul transferă execuția către o rutină de tratare a întreruperii (*Interrupt Service Routine*) aflată în memorie la o adresă configurată anterior; după terminarea acesteia se revine la execuția programului (similar cu un apel de funcție, dar inițiat de *hardware*)
- După ce procesorul tratează întreruperea, schimbând datele corespunzătoare cu perifericul, acesta eliberează linia de întrerupere

Ce probleme sunt dpdv electric?

- Întrucât întreruperile sunt inițiate de dispozitive, procesorul nu poate secvența accesul acestora la linia de întrerupere partajată, cum o face la liniile de date
- De ce nu se alocă o linie de întrerupere la fiecare dispozitiv?
  - se poate face și asta, dar nu când sunt foarte multe dispozitive (resursă limitată, precum magistrala de date)
- Trebuie utilizate ieșiri digitale ce **pot** fi active simultan
  - așa-numita structură ȘI-cablat (*wired-AND*)

Linie de întrerupere – ȘI cablat



Comentarii – ȘI cablat

- Se numește structură “ȘI-cablat” deoarece funcția booleană ȘI se realizează prin conectarea în paralel a ieșirilor cu drena în gol
  - când toate ieșirile semnalează logic 1, adică tranzistoarele sunt blocate, linia este în stare 1
  - când una sau mai multe ieșiri semnalează logic 0, adică se deschid unul sau mai multe tranzistoare, linia este trasă spre masă => stare logic 0

Comentarii – porți cu drena în gol

- Se numesc **porți cu drena în gol** (*open-drain*) sau **cu colectorul în gol** (*open-collector*) dacă erau realizate în tehnologie TTL
  - deoarece aceasta este structura ieșirii lor – drena/colectorul tranzistorului care trage ieșirea la masă nu are alte elemente conectate – se bazează pe faptul că va fi conectată o rezistență de *pull-up* în exterior pentru a asigura nivelul *high*
- Nu funcționează fără rezistența de *pull-up*
- Dezavantaj: comutarea L->H este lentă

Variante arhitecturale

- Calculatoarele contemporane au arhitecturi:
  - optimizate pentru performanță
    - legături punct-la-punct în loc de magistrale partajate
    - ex.: interfață directă procesor-memorie, canale *PCI Express* dedicate fiecărui periferic
    - aplicații: calculatore personale, servere
  - optimizate pentru cost / consum
    - arhitecturi integrate tip *microcontroller (MCU)* sau *System-on-a-Chip* (*SoC*) – includ procesorul, magistrala, memoria și perifericele într-un singur circuit integrat
    - aplicații: *smartphones*, *embedded systems, Internet of Things* dar și sisteme de control industriale, sisteme de monitorizare autonome, roboți etc.

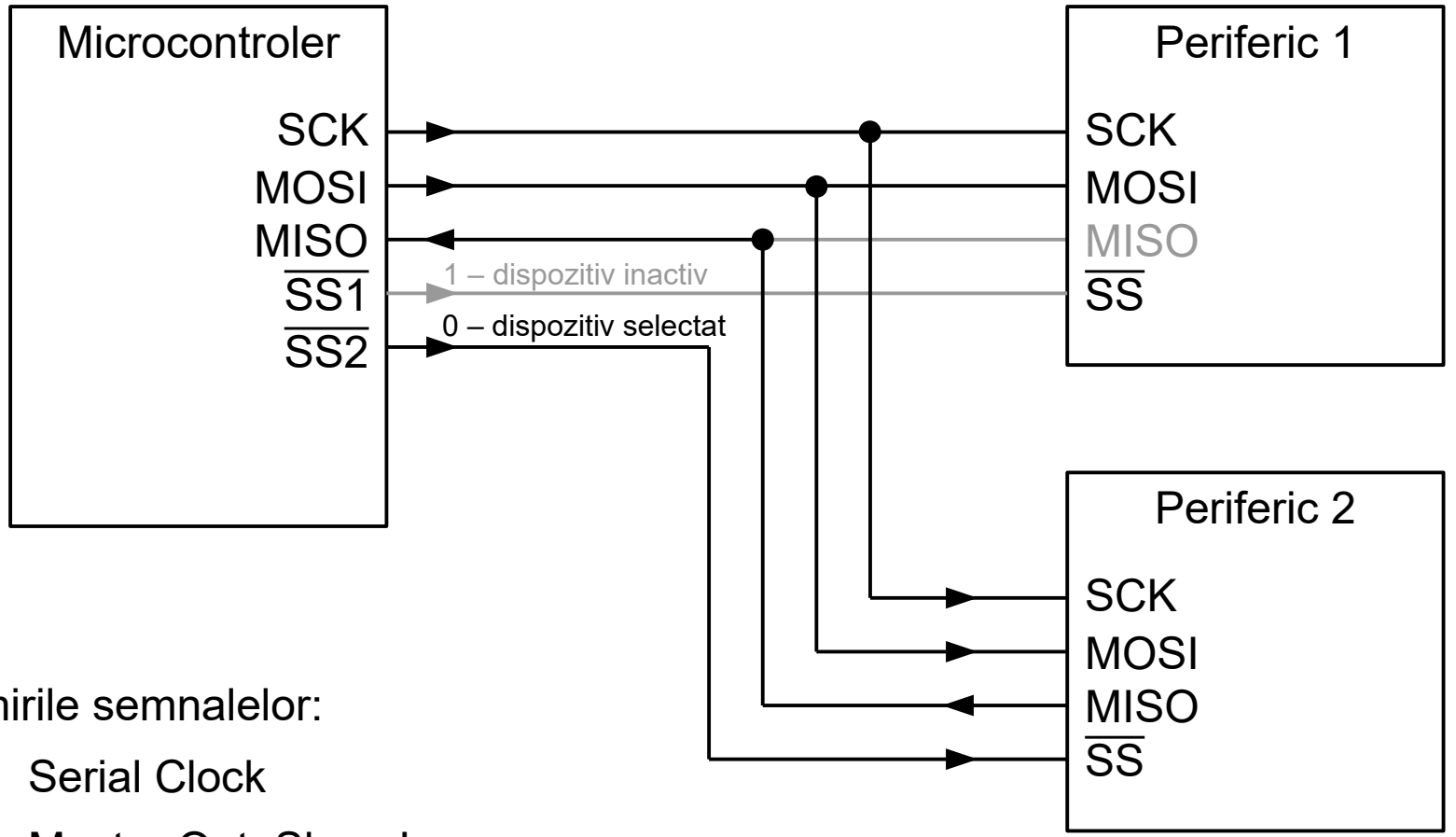
Aplicații *embedded*

- sisteme bazate pe **microcontroler**
- sisteme puternic integrate – magistralele discutate anterior sunt interne (inaccesibile)
- dotate cu magistrale externe specifice
  - optimizate pentru a fi compacte și cu cost redus
    - număr redus de semnale
    - de obicei **comunicație serială**
  - destinate interfațării de senzori, module de comunicație, ADC-uri, DAC-uri etc.
  - cele mai utilizate: **SPI** și **I^2C**

Magistrala SPI – descriere

- *Serial Peripheral Interface*
  - magistrală serială sincronă – transferul se realizează sincron cu un semnal de ceas; biții sunt transferați unul după celălalt (versus magistrala clasică unde sunt transferați în paralel – simultan)
  - dispozitivele trebuie selectate direct – nu există un mecanism prescris de adresare, ci o linie de selecție dedicată la fiecare dispozitiv
  - permite comunicație bidirecțională simultană – există două linii de date, una pentru fiecare direcție
  - dispozitivele utilizează o **ieșire cu 3 stări** pentru a transmite date pe linia partajată când sunt selectate

Magistrala SPI – exemplu



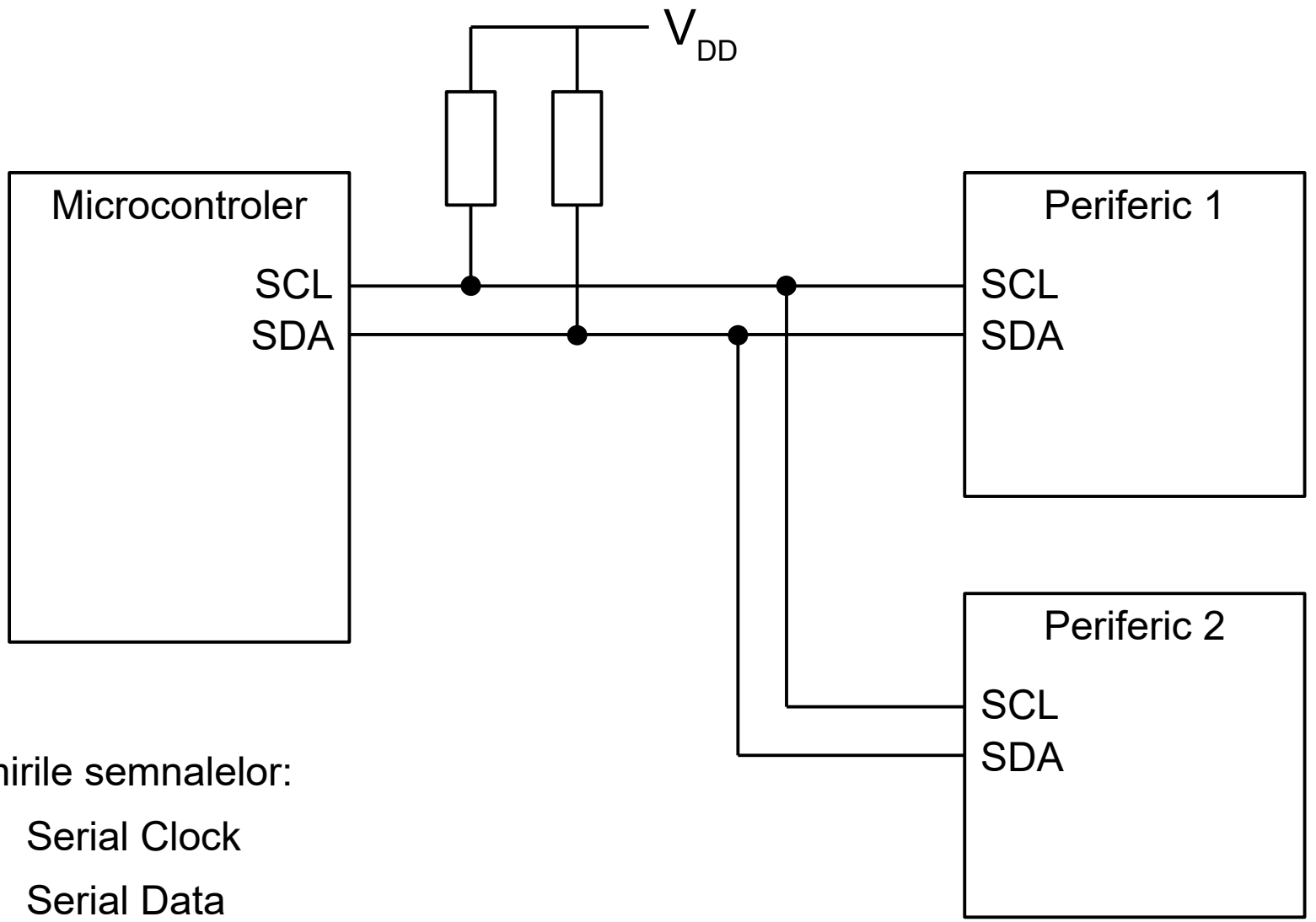
Denumirile semnalelor:

SCK	Serial Clock
MOSI	Master Out, Slave In
MISO	Master In, Slave Out
SS	Slave Select

Magistrala I^2C – descriere

- *Inter-Integrated Circuit*
  - magistrală serială sincronă
  - compactă – doar două semnale (ceas și date)
  - bidirecțională, dar nu simultan
  - protocol de comunicație complex
    - comunicația începe prin transmiterea adresei dispozitivului vizat – se elimină astfel nevoia de a avea linii de adresă / selecție dedicate
  - dispozitivele utilizează **ieșiri cu drena în gol** legate în paralel pe magistrală

Magistrala I²C – exemplu



Denumirile semnalelor:

SCL Serial Clock

SDA Serial Data